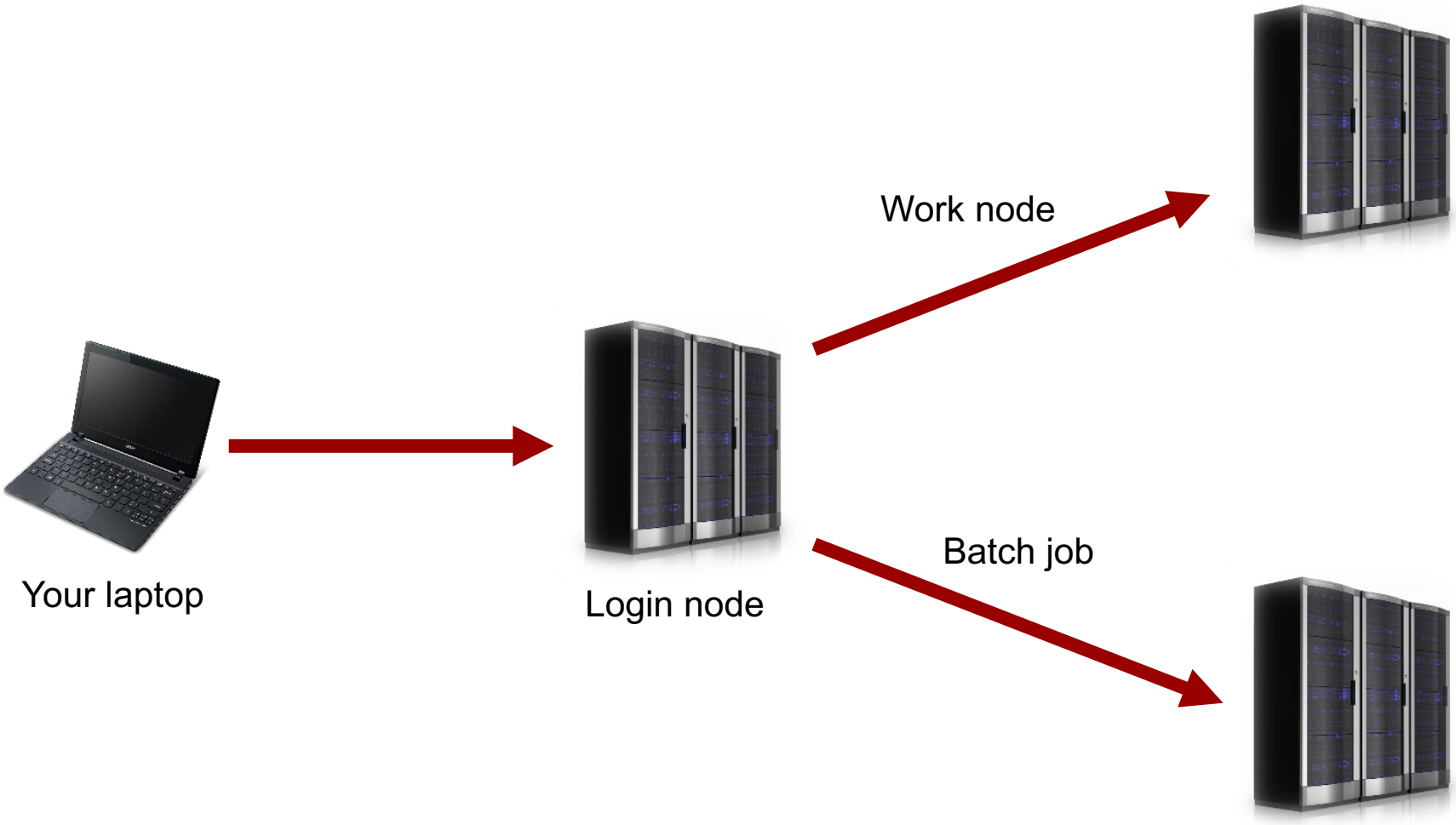
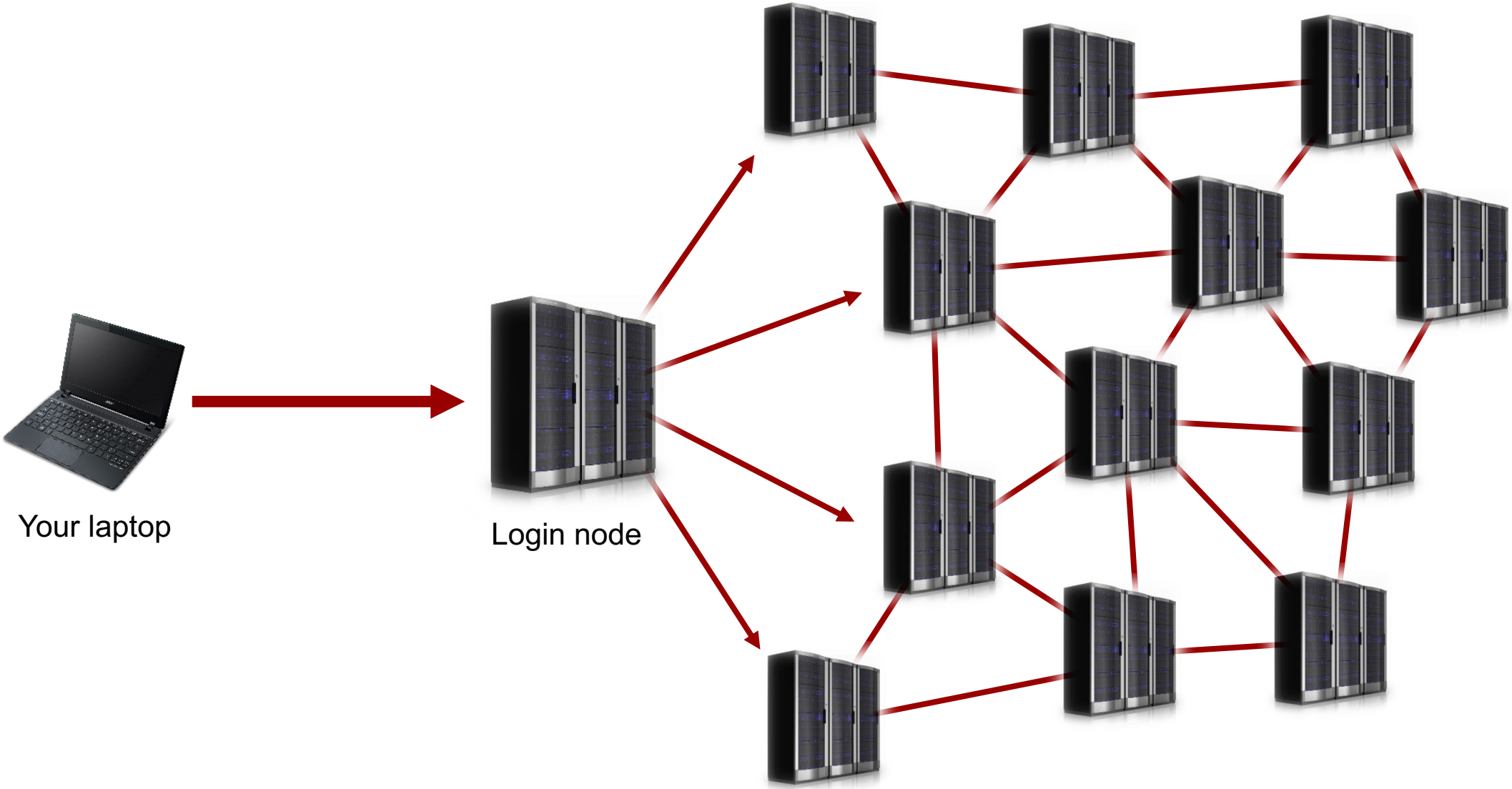
Week 11 – HPC Workflows

02613 Python and High-Performance Computing

So far: HPC as a bigger computer



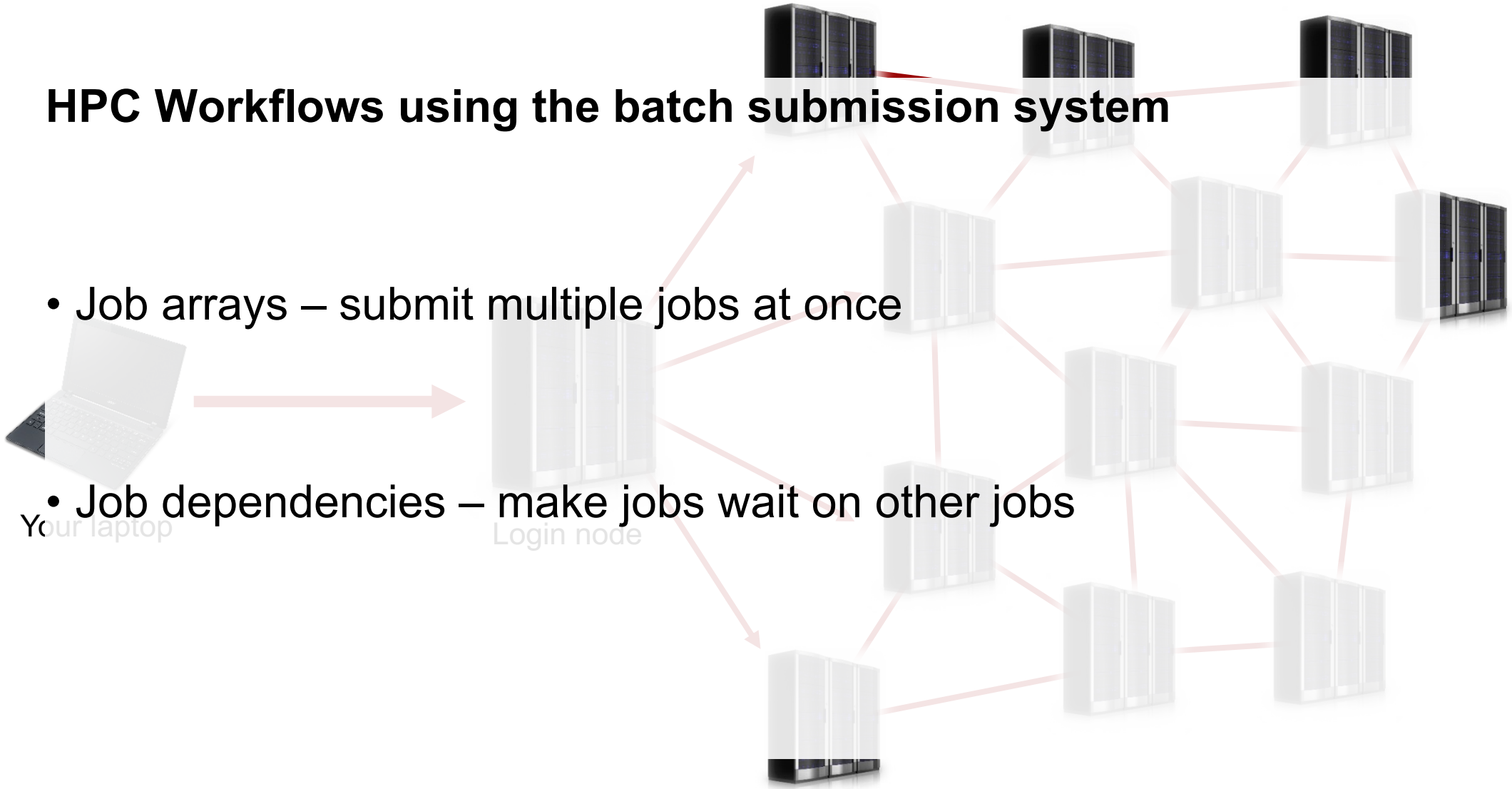
Today: HPC as a compute cluster



Today: HPC as a compute cluster

HPC Workflows using the batch submission system

- Job arrays – submit multiple jobs at once
- Job dependencies – make jobs wait on other jobs



Job scripts: revision

```
#!/bin/bash
#BSUB -J name                          Job name
#BSUB -q hpc                           Queue name
#BSUB -W 2                             Wall-clock time
#BSUB -R ”rusage[mem=512MB]”           Resources (mem)
#BSUB –n 4                             Number of cores
#BSUB –R ”span[hosts=1]”               All cores on same host
#BSUB -o name_%J.out                   Output file (stdout)
#BSUB -e name_%J.err                   Output file (stderr)

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python script.py input.npy 4
```

submit.sh

Job options

Job commands

```
$
```

Job scripts: revision

```
#!/bin/bash
#BSUB -J name                          Job name
#BSUB -q hpc                           Queue name
#BSUB -W 2                             Wall-clock time
#BSUB -R ”rusage[mem=512MB]”           Resources (mem)
#BSUB –n 4                             Number of cores
#BSUB –R ”span[hosts=1]”               All cores on same host
#BSUB -o name_%J.out                   Output file (stdout)
#BSUB -e name_%J.err                   Output file (stderr)

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python script.py input.npy 4
```

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bstat
JOBID  ...   QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name          0 PEND ...
$ bstat
JOBID  ...   QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name          4 RUN  ...
```

Job arrays

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python train.py --lr=1e-3
```

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
```

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python train.py --lr=1e-4
```

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <702577> is submitted to queue <hpc>.
```

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python train.py --lr=1e-5
```

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <702577> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <702578> is submitted to queue <hpc>.
```

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB
#BSUB
#BSUB
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python train.py --lr=1e-5
```

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <702577> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <702578> is submitted to queue <hpc>.
```

Tedious for many values

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input001.png
```

submit.sh

```
$ bsub < submit.sh
Job <702579> is submitted to queue <hpc>.
```

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input182.png
```

submit.sh

```
...
$ bsub < submit.sh
Job <704256> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <704257> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <704258> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <704259> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <704260> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <704261> is submitted to queue <hpc>.
```

Job arrays: motivation

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB
#BSU
#BSUE
#BSUE
#BSUE

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input182.png
```

submit.sh

```
...
$ bsub < submit.sh
Job <704256> is submitted to queue <hpc>.
$ bsub < submit.sh
$ bsub < submit.sh
Job <704260> is submitted to queue <hpc>.
$ bsub < submit.sh
Job <704261> is submitted to queue <hpc>.
```

Also: you just found an error.
Time to bkill…

Job arrays

Commands Options

"Job arrays are groups of jobs with the ***same*** executable and resource requirements, but ***different*** input files."

Options

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out
#BSUB -e name_%J.err
```

Commands

```
source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input001.png
```

Input files

Job arrays

Commands Options

"Job arrays are groups of jobs with the ***same*** executable and resource requirements, but ***different*** input files."

"Job arrays can be submitted, controlled, and monitored as a ***single unit*** or as individual jobs or groups of jobs."

Options

"Each job submitted from a job array ***shares the same job ID*** as the job array and are ***uniquely referenced using an array index***."

```
#!/bin/bash
#BSUB -n 1
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -J name[1-10]
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input001.png
```

Commands

Input files

Job arrays

```
#!/bin/bash
#BSUB -J name[1-182]
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input001.png
```

Job array with indices from 1 to 182

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bstat
JOBID  USER  QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name[1]       4 PEND ...
702576 ...   hpc   name[2]       4 PEND ...
702576 ...   hpc   name[3]       4 PEND ...
702576 ...   hpc   name[4]       4 PEND ...
702576 ...   hpc   name[5]       4 PEND ...
702576 ...   hpc   name[6]       4 PEND ...
702576 ...   hpc   name[7]       4 PEND ...
702576 ...   hpc   name[8]       4 PEND ...
...
702576 ...   hpc   name[182]     4 PEND ...
```

Job arrays

“With great power…
comes great responsibility!”

- Uncle Ben, Spiderman

“So please, think before you submit large job arrays”

- Patrick

Job arrays

```
#!/bin/bash
#BSUB -J name[1-182]
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py input001.png
```

Job array with indices from 1 to 182

submit.sh

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bstat
JOBID  USER  QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name[1]      4 PEND ...
702576 ...   hpc   name[2]      4 PEND ...
702576 ...   hpc   name[3]      4 PEND ...
702576 ...   hpc   name[4]      4 PEND ...
702576 ...   hpc   name[5]      4 PEND ...
702576 ...   hpc   name[6]      4 PEND ...
702576 ...   hpc   name[7]      4 PEND ...
702576 ...   hpc   name[8]      4 PEND ...
...
702576 ...   hpc   name[182]    4 PEND ...
```

Same!

Index

Job arrays

```
#!/bin/bash
#BSUB -J name[1-182]
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py $LSB_JOBINDEX
```

submit.sh

Job array with indices from 1 to 182

Array index (will be 1, 2, 3, …, 182)

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bstat
JOBID  USER  QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name[1]       4 PEND ...
             name[2]       4 PEND ...
             name[3]       4 PEND ...
             name[4]       4 PEND ...
             name[5]       4 PEND ...
702576 ...   hpc   name[6]       4 PEND ...
702576 ...   hpc   name[7]       4 PEND ...
702576 ...   hpc   name[8]       4 PEND ...
...
702576 ...   hpc   name[182]     4 PEND ...
```

Easiest method:

Convert index to file name, parameter value, whatever, in the Python script.

Job arrays

```
#BSUB -J name[1-10]
```

Job array with indices
1, 2, 3, 4, 5, 6, 7, 8, 9, 10

```
#BSUB -J name[1,2,3,4,5,6,7,8,9,10]
```

Job array with indices
1, 2, 3, 4, 5, 6, 7, 8, 9, 10

start end step

```
#BSUB -J name[1-10:2]
```

Job array with indices
1, 3, 5, 7, 9

```
#BSUB -J name[1,4,10-14,20-30:2]
```

Job array with indices
1, 4, 10, 11, 12, 13, 14, 20, 22, 24, 26, 28, 30

Job arrays

```
#!/bin/bash
#BSUB -J name[1-182]
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py $LSB_JOBINDEX
```

submit.sh

All output in name_702576.out
All error messages in name_702576.err

Job ID

```
...
...mitted to queue <hpc>.
...
JOBID  USER  QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name[1]       4 PEND ...
702576 ...   hpc   name[2]       4 PEND ...
702576 ...   hpc   name[3]       4 PEND ...
702576 ...   hpc   name[4]       4 PEND ...
702576 ...   hpc   name[5]       4 PEND ...
702576 ...   hpc   name[6]       4 PEND ...
702576 ...   hpc   name[7]       4 PEND ...
702576 ...   hpc   name[8]       4 PEND ...
...
702576 ...   hpc   name[182]     4 PEND ...
```

Job arrays

```
#!/bin/bash
#BSUB -J name[1-182]
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512M
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J_%I.out
#BSUB -e name_%J_%I.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python preprocess.py $LSB_JOBINDEX
```

Job array index

submit.sh

Job 1 output in name_702576_1.out
Job 1 error messages in name_702576_1.err
Job 2 output in name_702576_2.out
Job 2 error messages in name_702576_2.err
...

```
d to queue <hpc>.

AME NALLOC STAT ...
1]       4 PEND ...
2]       4 PEND ...
702576 ...   hpc   name[3]       4 PEND ...
702576 ...   hpc   name[4]       4 PEND ...
702576 ...   hpc   name[5]       4 PEND ...
702576 ...   hpc   name[6]       4 PEND ...
702576 ...   hpc   name[7]       4 PEND ...
702576 ...   hpc   name[8]       4 PEND ...
...
702576 ...   hpc   name[182]     4 PEND ...
```

Monitoring job arrays

```
$ bstat
21415157   patmjen hpc        name[1]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[2]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[3]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[4]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[5]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[6]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[7]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[8]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[9]        0 PEND       -          0:00:00
21415157   patmjen hpc        name[10]       0 PEND       -          0:00:00
...
21415157   patmjen hpc        name[182]      0 PEND       -          0:00:00
```

Monitoring job arrays

```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER    NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
```

Job ID

Job name / array specification

User

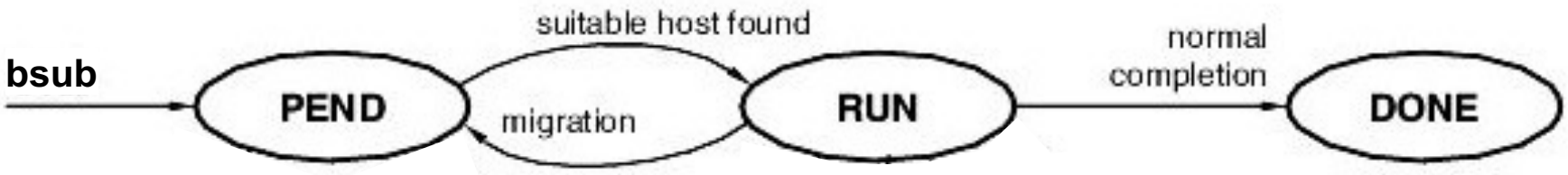
Num. jobs

Job states

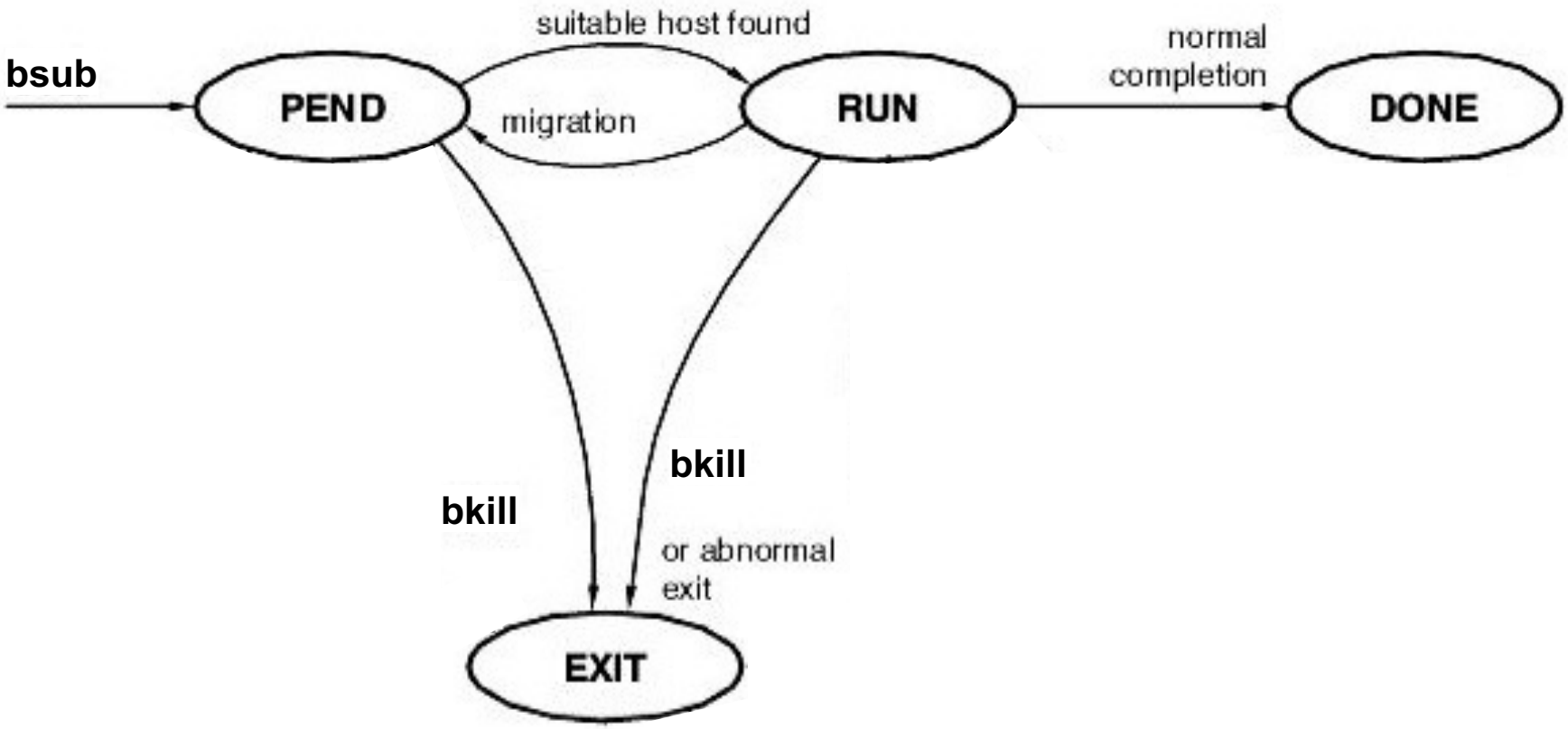
Monitoring job arrays

```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10    0    0   10    0     0     0     0
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10    0    3    7    0     0     0     0
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10    0   10    0    0     0     0     0
```

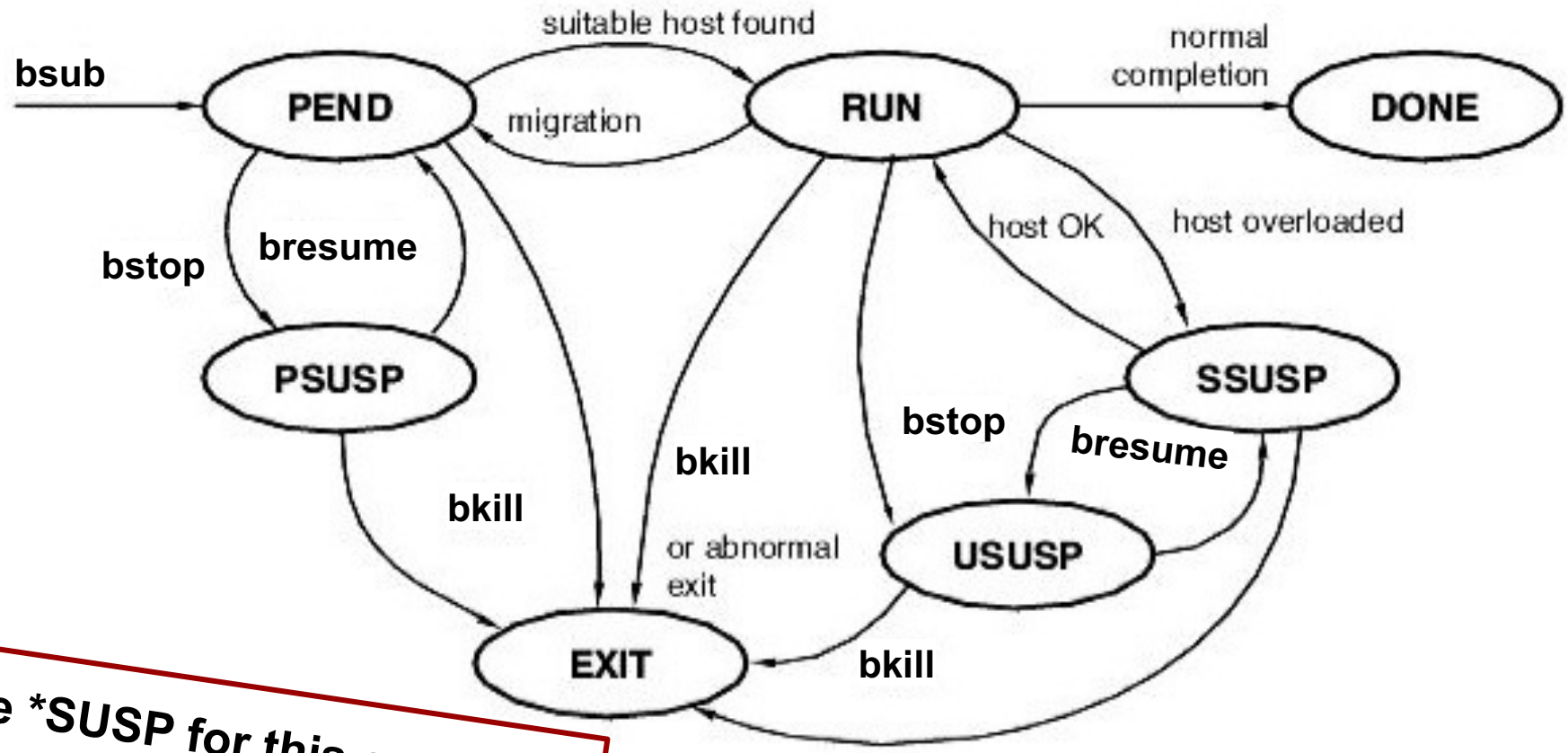
Job states


bsub
PEND
suitable host found
migration
RUN
normal
completion
DONE

Job states


bsub
PEND
suitable host found
migration
RUN
normal
completion
DONE
bkill
bkill
or abnormal
exit
EXIT

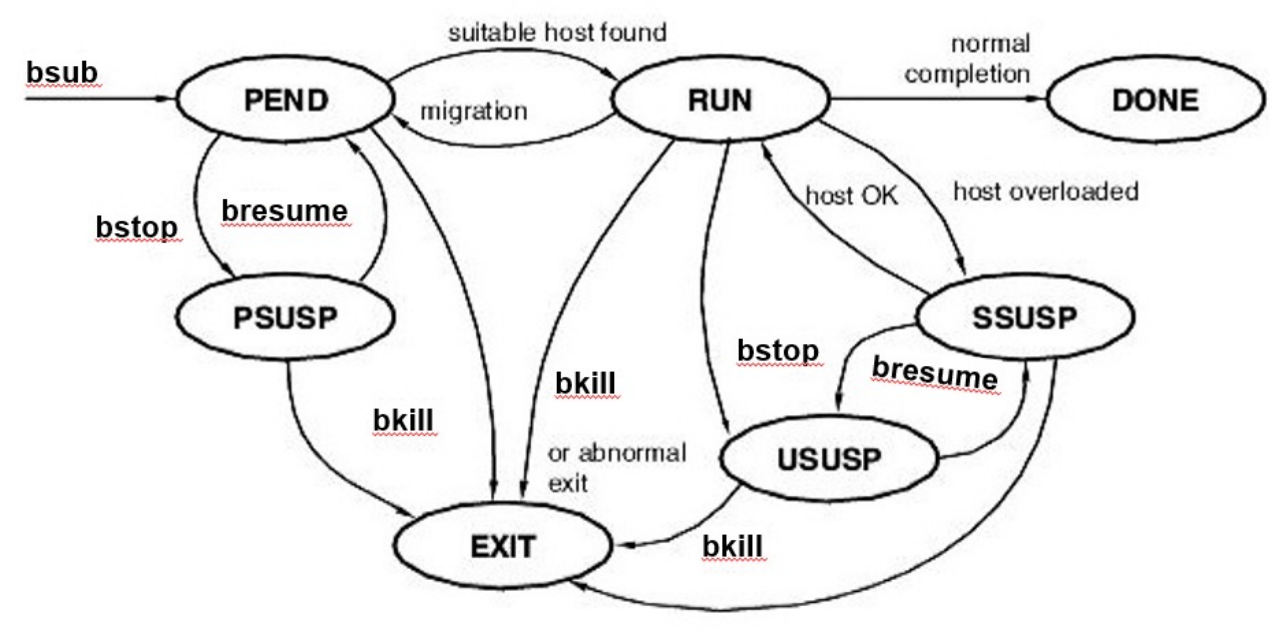
Job states



Can ignore *SUSP for this course

Monitoring job arrays

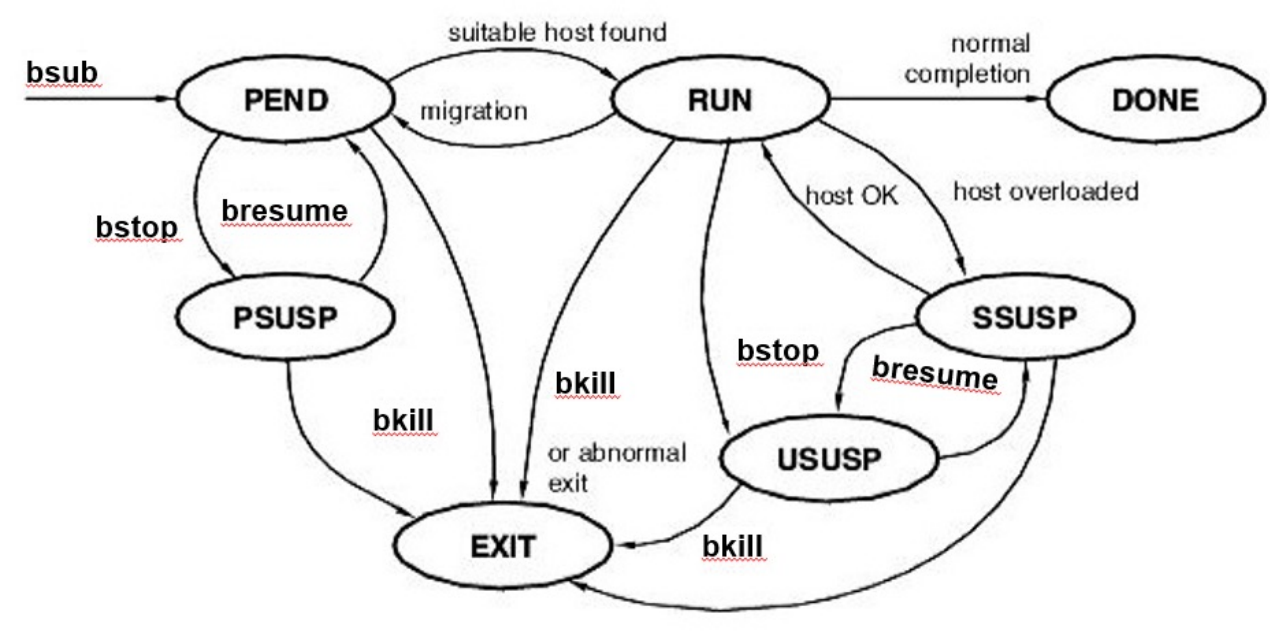
```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
$ bjobs -A
                                       DONE  RUN EXIT SSUSP USUSP PSUSP
                                          0   10    0     0     0     0
                                       DONE  RUN EXIT SSUSP USUSP PSUSP
                                          3    7    0     0     0     0
                                       DONE  RUN EXIT SSUSP USUSP PSUSP
                                         10    0    0     0     0     0
```



Monitoring job arrays

```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
$ bjobs -A
                                          DONE  RUN EXIT SSUSP USUSP PSUSP
                                             0   10    0     0     0     0
                                          DONE  RUN EXIT SSUSP USUSP PSUSP
                                             3    7    0     0     0     0
                                          DONE  RUN EXIT SSUSP USUSP PSUSP
                                             6    0    4     0     0     0
```

4 out of 10 jobs crashed



Peeking job arrays

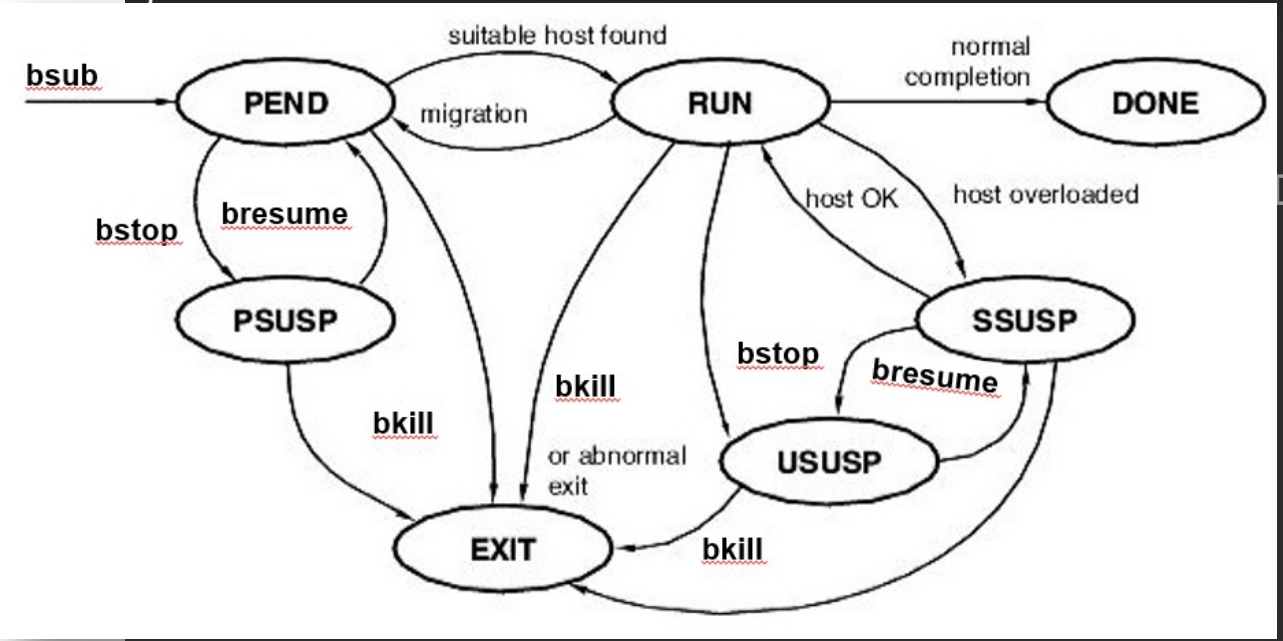
```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10    0    0   10    0     0     0     0
$ bpeek 21415299[3]
<< output from stdout >>

<< output from stderr >>
```

Must specify job index

Killing job arrays

```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
$ bkill 21415299
```




```
DONE  RUN EXIT SSUSP USUSP PSUSP
   0    0   10     0     0     0
```

10 out of 10 jobs killed

Killing job arrays

```
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10   10    0    0    0     0     0     0
$ bkill 21415299[1-5]
Job <21415299[1-5:1]>: Operation is in progress
$ bjobs -A
JOBID       ARRAY_SPEC  OWNER   NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
21415299    name[1-1  patmjen      10    0    5    0    5     0     0     0
```

5 out of 10 jobs finished successfully

5 out of 10 jobs killed

Job arrays and e-mail notification

```
#!/bin/bash
#BSUB -J name[1-182]
#BSUB -q hpc
#BSUB -W 2
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 4
#BSUB –R ”span[hosts=1]”
#BSUB -o name_%J.out
#BSUB -e name_%J.err
#BSUB –B s123456@student.dtu.dk
#BSUB –N s123456@student.dtu.dk
source /dtu/projects/02613_2025/
conda activate 02613

python preprocess.py $LSB_JOBINDEX
```

submit.sh

Job array with indices from 1 to 182

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bstat
JOBID  USER  QUEUE JOB_NAME NALLOC STAT ...
702576 ...   hpc   name[1]       4 PEND ...
             name[2]       4 PEND ...
             name[3]       4 PEND ...
             name[4]       4 PEND ...
             name[5]       4 PEND ...
             name[6]       4 PEND ...
             name[7]       4 PEND ...
             name[8]       4 PEND ...

             name[182]     4 PEND ...
```

Warning:

Using notifications in job arrays can generate many e-mails, i.e. 2x 182 in this case! The DTU mail server rejects e-mails, if too many (>100) arrive in a short interval!

So please, do **not** use notifications in job arrays! Thanks!

Job dependencies

Job dependencies: motivation

”Sometimes, whether a job should start ***depends*** on the result of another job”

Examples:

- GPU jobs have a time limit of 24 hours
  Schedule job to resume processing after current one times out

- GPU pre-processing followed by long CPU analysis
  Schedule GPU part as one job and CPU analysis as a depending job

- Process many files independently followed by summary analysis
  Schedule job array for files and summary analysis as depending job

Job dependencies

```
#!/bin/bash
#BSUB -J job1
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job1_%J.out
#BSUB -e job1_%J.err

sleep 90
echo ”Some text” > job.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err


cat job.data
```

job2.sh

depends on

Job dependencies

```
#!/bin/bash
#BSUB -J job1
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job1_%J.out
#BSUB -e job1_%J.err

sleep 90
echo ”Some text” > job.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w job1
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err

cat job.data
```

Wait for job with name ”job1” to finish successfully

job2.sh

depends on

Job dependencies

```
#!/bin/bash
#BSUB -J job1
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job1_%J.out
#BSUB -e job1_%J.err

sleep 90
echo ”Some text” > job.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w job1
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err

cat job.data
```

Wait for job with name ”job1” to finish successfully

job2.sh

```
$ bsub < job1.sh
Job <21416739> is submitted to queue <hpc>.
$ bsub < job2.sh
Job <21416740> is submitted to queue <hpc>.
```

Job dependencies

```
#!/bin/bash
#BSUB -J job1
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job1_%J.out
#BSUB -e job1_%J.err

sleep 90
echo ”Some text” > job.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w job1
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err

cat job.data
```

Wait for job with name ”job1” to finish successfully

job2.sh

```
$ bstat
JOBID      USER    QUEUE      JOB_NAME   NALLOC STAT  START_TIME      ELAPSED
21416739   patmjen hpc        job1            1 RUN   Apr 15 18:19    0:00:51
21416740   patmjen hpc        job2            0 PEND       -          0:00:00
```

Job dependencies

```
#!/bin/bash
#BSUB -J job1
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job1_%J.out
#BSUB -e job1_%J.err

sleep 90
echo ”Some text” > job.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w job1
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err

cat job.data
```

Wait for job with name ”job1” to finish successfully

job2.sh

```
$ bstat
JOBID      USER    QUEUE      JOB_NAME   NALLOC STAT  START_TIME      ELAPSED
21416740   patmjen hpc        job2            1 RUN   Apr 15 18:21    0:00:00
```

Job dependencies

```
#!/bin/bash
#BSUB -J job1
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job1_%J.out
#BSUB -e job1_%J.err

sleep 90
echo ”Some text” > job.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w 21416739
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err

cat job.data
```

Wait for job with ID 21416739 to finish successfully

job2.sh

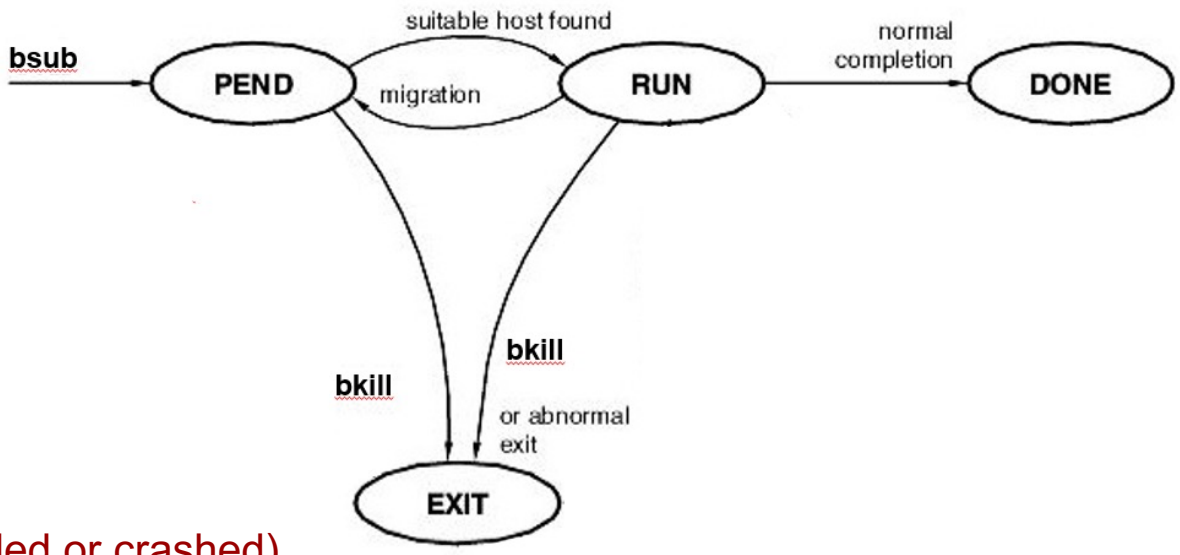
```
$ bsub < job1.sh
Job <21416739> is submitted to queue <hpc>.
$ bsub < job2.sh
Job <21416740> is submitted to queue <hpc>.
```

Job dependencies

```
#BSUB -w job1
```

Wait for job with name ”job1” to finish **successfully**

Job dependencies




```
#BSUB -w job1
```

Wait for job with name ”job1” to reach state DONE

```
#BSUB -w done(job1)
```

Wait for job with name ”job1” to reach state DONE

```
#BSUB -w exit(job1)
```

Wait for job with name ”job1” to reach state EXIT (killed or crashed)

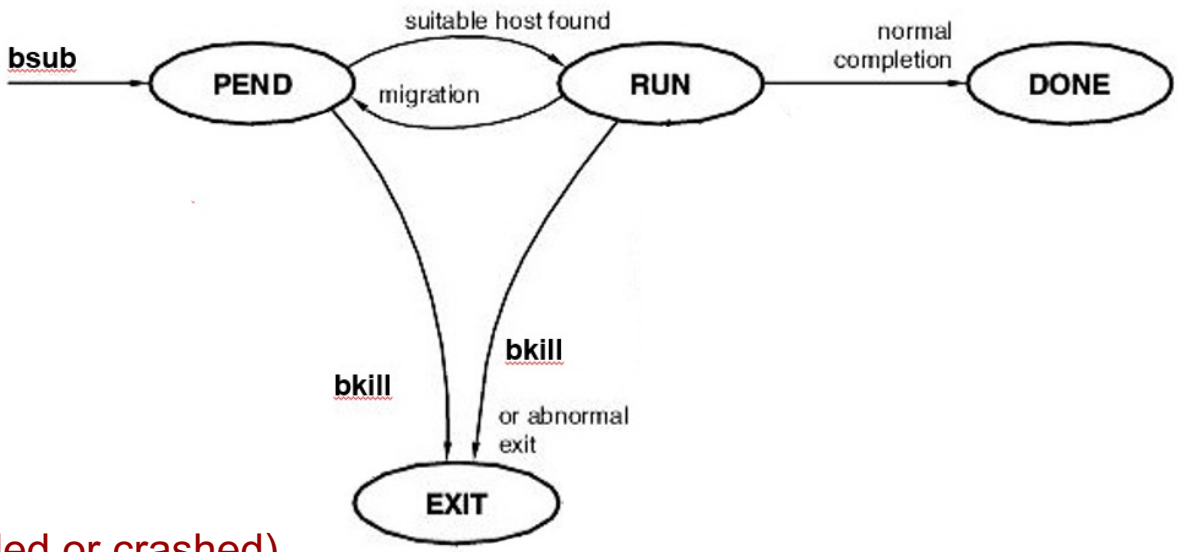
```
#BSUB -w ended(job1)
```

Wait for job with name ”job1” to reach state EXIT or DONE (finished for any reason)

```
#BSUB -w started(job1)
```

Wait for job with name ”job1” to reach state RUN, EXIT or DONE (started)

Job dependencies




```
#BSUB -w 1234567
```

Wait for job with name ”job1” to reach state DONE

```
#BSUB -w done(1234567)
```

Wait for job with name ”job1” to reach state DONE

```
#BSUB -w exit(1234567)
```

Wait for job with name ”job1” to reach state EXIT (killed or crashed)

```
#BSUB -w ended(1234567)
```

Wait for job with name ”job1” to reach state EXIT or DONE (finished for any reason)

```
#BSUB -w started(1234567)
```

Wait for job with name ”job1” to reach state RUN, EXIT or DONE (started)

Job dependencies for job arrays

```
#!/bin/bash
#BSUB -J array[1-10]
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o array_%J_%I.out
#BSUB -e array_%J_%I.err

sleep 90
echo ”Some text” > job_$LSB_JOBINDEX.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err


cat job_*.data
```

job2.sh

depends on all jobs in array

Job dependencies for job arrays

```
#!/bin/bash
#BSUB -J array[1-10]
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o array_%J_%I.out
#BSUB -e array_%J_%I.err

sleep 90
echo ”Some text” > job_$LSB_JOBINDEX.data
```

job1.sh

```
#!/bin/bash
#BSUB -J job2
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w array
#BSUB -o job2_%J.out
#BSUB -e job2_%J.err

cat job_*.data
```

job2.sh

Wait for job with name ”array” to finish successfully
Since the job is an array, wait for whole array

depends on all jobs in array

Job dependencies for job arrays

```
#!/bin/bash
#BSUB -J array1[1-10]
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o array1_%J_%I.out
#BSUB -e array1_%J_%I.err

sleep 90
echo ”Some text” > job_$LSB_JOBINDEX.data
```

job1.sh

```
#!/bin/bash
#BSUB -J array2[1-10]
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o array2_%J_%I.out
#BSUB -e array2_%J_%I.err



cat job_$LSB_JOBINDEX.data
```

job2.sh

depends on *corresponding* jobs in array

Job dependencies for job arrays

```
#!/bin/bash
#BSUB -J array1[1-10]
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB -o array1_%J_%I.out
#BSUB -e array1_%J_%I.err

sleep 90
echo ”Some text” > job_$LSB_JOBINDEX.data
```

job1.sh

```
#!/bin/bash
#BSUB -J array2[1-10]
#BSUB -q hpc
#BSUB -W 5
#BSUB -R ”rusage[mem=512MB]”
#BSUB –n 1
#BSUB –w done(array1[*])
#BSUB -o array2_%J_%I.out
#BSUB -e array2_%J_%I.err

cat job_$LSB_JOBINDEX.data
```

job2.sh

Wait for corresponding job in array to be DONE. Arrays must have same length!

depends on *corresponding* jobs in array

More options available – see documentation

IBM | Documentation | Search in IBM Spectrum LSF 10.1.0

IBM Spectrum LSF

Change version

10.1.0

Show full table of contents

Filter on titles

- -w
- -wa
- -wt
- -XF
- -x
- -yaml
- -Zs
- command

Download PDF

All products / IBM Spectrum LSF / 10.1.0 /

Was this topic helpful?

-W

Last Updated: 2024-01-08

LSF does not place your job unless the dependency expression evaluates to TRUE.

Categories

schedule

Synopsis

bsub -w '*dependency_expression*'[**-ti**]

Description

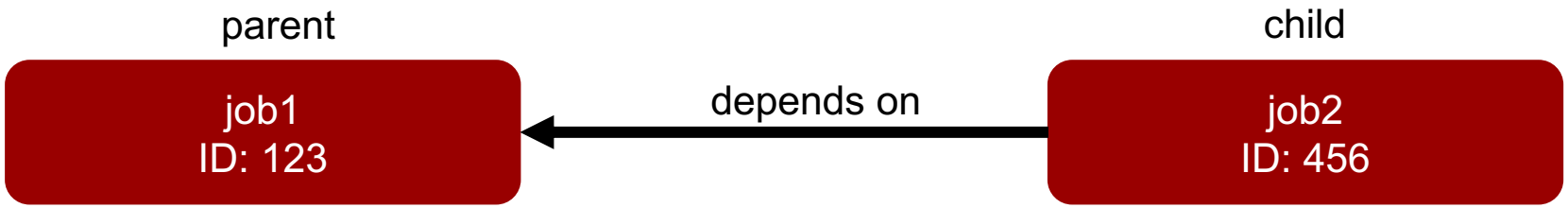
-w '***dependency_expression***' [**-ti**]

If you specify a dependency on a job that LSF cannot find (such as a job that has not

On this page

- Categories
- Synopsis
- Description

Monitoring dependencies

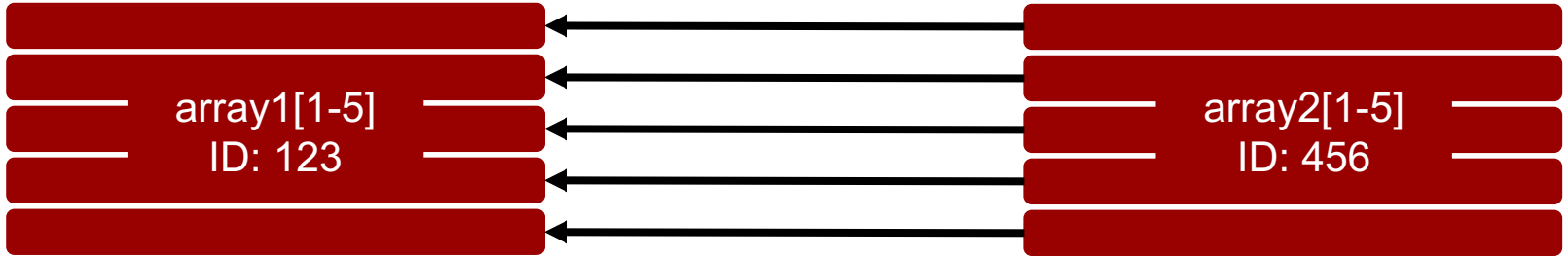



```
$ bjdepinfo 456
JOBID          PARENT         PARENT_STATUS  PARENT_NAME  LEVEL
456            123            RUN            job1         1
$ bjdepinfo 123
Job <123> does not depend on other jobs.
$ bjdepinfo -c 123
JOBID          CHILD          CHILD_STATUS  CHILD_NAME  LEVEL
123            456            PEND          job2        1
```

Monitoring dependencies

```
$ bjdepinfo 789
JOBID           PARENT          PARENT_STATUS   PARENT_NAME   LEVEL
789             456             PEND            job2          1
$ bjdepinfo -r2 789
JOBID           PARENT          PARENT_STATUS   PARENT_NAME   LEVEL
789             456             PEND            job2          1
456             123             RUN             job1          2
```

Monitoring dependencies




```
$ bjdepinfo 456
JOBID          PARENT         PARENT_STATUS  PARENT_NAME   LEVEL
456[1]         123[1]         RUN            array1[1]     1
456[2]         123[2]         RUN            array1[2]     1
456[3]         123[3]         RUN            array1[3]     1
456[4]         123[4]         RUN            array1[4]     1
456[5]         123[5]         RUN            array1[5]     1
$ bjdepinfo 456[2]
JOBID          PARENT         PARENT_STATUS  PARENT_NAME   LEVEL
456[2]         123[2]         RUN            array1[1]     1
```

Summary

Before: solving single tasks

Now: building pipelines

Job arrays: doing the same thing for many inputs

Job dependencies: chaining multiple jobs

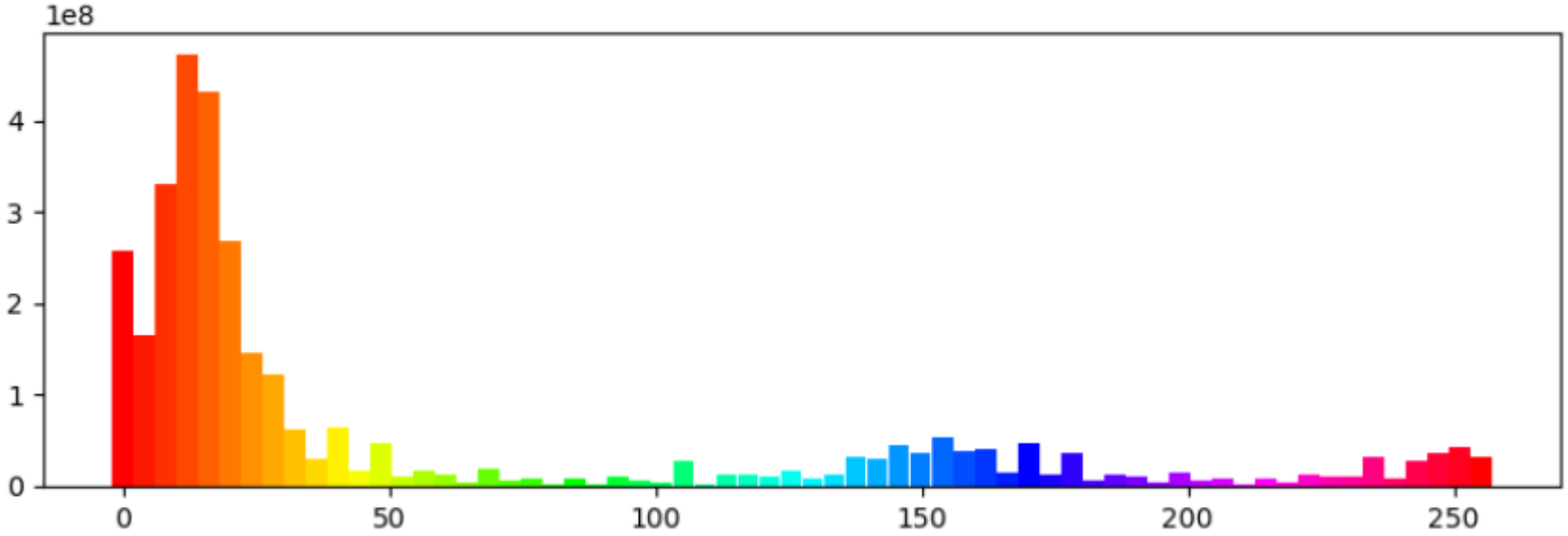
Today’s exercise

HPC workflows

- Warmup with job scripts
- Processing pipeline for face pictures

HPC workflows

Compute color (hue) histogram for all pictures



HPC workflows

Compute color (hue) histogram for all pictures

1. Compute sub-histogram for each folder (job array)
2. Collect sub-histogram to final histogram (dependant job)

```
$ ls /dtu/projects/02613_2025/data/celeba/images/
000000  008000  016000  024000  032000  040000  048000  056000  064000  072000
080000  088000  096000  104000  112000  120000  128000  136000  144000  152000
...
$ ls /dtu/projects/02613_2025/data/celeba/images/000000
000000.jpg  000059.jpg  000118.jpg  000177.jpg  000236.jpg  000295.jpg  000354.jpg
000413.jpg  000472.jpg  000531.jpg  000590.jpg  000649.jpg  000708.jpg  000767.jpg
...
```

Useful concepts

Job arrays
#BSUB -J array[1-5]
#BSUB -J array[1-5:2]
#BSUB -J array[1,2,3,4,5]

bjobs -A

Job dependencies
#BSUB -w jobname
#BSUB -w 123456
#BSUB -w done(jobname)
#BSUB -w done(array[*])

bjdepinfo 123456
bjdepinfo -c 123456
bjdepinfo 123456[2]

Change to work node
linuxsh

Submit job script
bsub < submit.sh

Job status
bstat / bjobs

Check job output
bpeek 123456
bpeek 123456[2]

Kill job
bkill <JOBID>